



BAHRIA UNIVERSITY KARACHI CAMPUS

Department of Software Engineering

COURSE: CEL 220
COMPUTER ARCHITECTURE & LOGIC DESIGN LAB
PROJECT PROPOSAL
CLASS: BSE –3B (FALL - 2025)

MIPS-BASED DIGITAL LOGIC AND PROCESSOR SIMULATOR

Group Members

S. No.	Student Name	Enrollment Number
1	Arbaz Khan Orakzai (Team Lead)	02-131232-110
2	Abdul Rehman	02-131242-101
3	Rana Mudassir Ahsan	02-131242-030
4	Muhammad Ahmed	02-131242-115

Submitted to:

Course Instructor: Engr. Ramsha Mashood
Lab Instructor: Engr. Noor us Sabah

Date: 20-November-2025

1. Executive Summary

This project presents a comprehensive hierarchical simulation of digital logic design and processor architecture implemented entirely in MIPS Assembly Language. The simulator progressively builds computational complexity from fundamental logic gates through arithmetic circuits to a functional Von Neumann architecture processor, demonstrating the foundational principles of computer organization and digital systems design.

The implementation encompasses four progressive phases: basic gate-level logic verification, combinational circuit construction through adder designs, an 8-bit Arithmetic Logic Unit with status flag generation, and culminates in a simplified MIPS-inspired processor with instruction fetch-decode-execute cycles. This hierarchical approach mirrors real-world processor design methodology while addressing the complex engineering problem of hardware-software co-design within the constraints of assembly language programming.

2. Problem Statement

Modern computer architecture education faces a pedagogical challenge: students often encounter processor design concepts abstractly without understanding the underlying computational primitives that enable instruction execution. Traditional approaches either focus exclusively on hardware simulation tools or high-level software implementations, creating a gap in understanding how software orchestrates hardware behavior at the lowest abstraction level.

This project addresses the complex engineering problem of simulating digital logic circuits and a functional processor using only MIPS Assembly Language, demonstrating how software can model hardware behavior while highlighting the intrinsic relationship between instruction set architectures and their underlying digital logic implementations. The challenge lies in implementing stateful hardware components, memory hierarchies, and control flow mechanisms using only register-based operations and syscalls available in MIPS assembly.

3. Project Objectives

3.1 Primary Objectives

1. Implement comprehensive truth table verification for fundamental logic gates (AND, OR, NOT) using bitwise MIPS instructions
2. Design and verify combinational arithmetic circuits including half adders and full adders with carry propagation logic
3. Develop an 8-bit Arithmetic Logic Unit supporting multiple operations (ADD, SUB, AND, OR) with condition code flag generation
4. Create a functional mini-CPU simulator with instruction memory, data memory, register file, and fetch-decode-execute control logic

3.2 Learning Outcomes

- Demonstrate mastery of MIPS assembly language including control flow, subroutine management, and memory organization
- Apply theoretical knowledge of digital logic design and computer architecture to practical software implementation
- Develop systematic testing methodologies for hardware simulation verification
- Understand the hierarchical nature of computer systems from transistor-level logic to instruction set architectures

4. Technical Approach and Methodology

4.1 System Architecture Overview

The simulator employs a bottom-up construction methodology, beginning with atomic logic operations and progressively composing them into increasingly complex computational structures. Each phase builds upon verified components from previous phases, ensuring correctness through hierarchical validation. The architecture follows a modular design pattern with clearly defined interfaces between simulation layers.

4.2 Phase-by-Phase Implementation Strategy

Phase 1: Logic Gate Simulation

Implementation of fundamental Boolean operations using MIPS bitwise instructions (AND, OR, XOR). Each gate generates comprehensive truth tables through exhaustive input enumeration, verifying correct logical behavior for all possible input combinations. The output format presents inputs and corresponding outputs in tabular form for easy verification.

Phase 2: Combinational Arithmetic Circuits

Construction of half adder and full adder circuits using previously verified gate primitives:

- Half Adder: Implements sum ($A \oplus B$) and carry ($A \text{ AND } B$) generation
- Full Adder: Extends half adder with carry-in support, demonstrating carry propagation logic essential for multi-bit arithmetic

Phase 3: 8-Bit Arithmetic Logic Unit

Design of a parameterized ALU supporting multiple arithmetic and logical operations with condition code generation:

- Operation Set: ADD (arithmetic addition), SUB (arithmetic subtraction), AND (bitwise conjunction), OR (bitwise disjunction)
- Flag Generation: Zero flag (result == 0), Negative flag (bit 7 set) for conditional branch support
- Comprehensive test vectors demonstrating correct operation across diverse input combinations including edge cases

Phase 4: Mini-CPU Processor Simulation

Implementation of a Von Neumann architecture processor with the following subsystems:

- Register File: Four 8-bit general-purpose registers (R0-R3) with R0 hardwired to zero following MIPS convention
- Data Memory: 256-byte byte-addressable memory space for variable storage and manipulation
- Instruction Memory: Word-organized program storage with 4-word instruction encoding (opcode, destination, source1, source2/immediate)
- Instruction Set: Arithmetic (ADD, SUB), logical (AND, OR), memory (LOAD, STORE), and control (HALT) operations
- Control Unit: Fetch-decode-execute cycle with program counter management and instruction dispatch logic

The demonstration program showcases a complete computation: loading operands from memory, performing arithmetic operations, and storing results back to memory, illustrating the full datapath and control flow of a simplified processor.

4.3 Verification and Validation Strategy

- Exhaustive Testing: Generate complete truth tables for all combinational logic to verify correctness
- Boundary Analysis: Test arithmetic operations with edge cases (maximum values, zero operands, overflow conditions)
- Hierarchical Validation: Each phase depends only on verified components from previous phases
- State Inspection: CPU simulator displays register file and memory contents after each instruction for debugging and verification

5. Module Distribution and Responsibilities

The project is divided into specialized modules based on team members' technical strengths and interests. Each module represents a critical component of the overall system with clearly defined interfaces and deliverables.

Module	Assigned To	Phase	Key Components
Module 1	Arbaz Khan Orakzai	Phase 3 & 4	ALU Design & CPU Architecture
Module 2	Abdul Rehman	Phase 1	Logic Gate Implementation
Module 3	Rana Muddasir	Phase 2	Adder Circuits & Testing
Module 4	Muhammad Ahmed	All Phases	System Integration & I/O

5.1 Module 1: ALU Design and CPU Architecture

Assigned to: Arbaz Khan Orakzai

This module focuses on the most architecturally complex components, requiring deep understanding of processor design principles and sophisticated control flow management.

Responsibilities:

- Design and implement the 8-bit ALU with support for arithmetic (ADD, SUB) and logical (AND, OR) operations
- Implement condition code flag generation logic (Zero flag, Negative flag) for conditional operations
- Architect the mini-CPU control unit including fetch-decode-execute cycle implementation
- Design instruction encoding scheme and instruction memory organization
- Implement register file with read/write operations and R0 hardwiring logic
- Develop instruction dispatcher and opcode execution logic

Technical Justification:

This assignment leverages advanced pattern recognition and systems-level thinking capabilities. The ALU requires careful state management and operation multiplexing, while the CPU architecture demands understanding of datapath-controller interaction and instruction cycle orchestration.

5.2 Module 2: Logic Gate Implementation

Assigned to: Abdul Rehman

This module establishes the foundational layer of the simulation, implementing atomic Boolean operations that serve as building blocks for all subsequent phases.

Responsibilities:

- Implement AND gate truth table generation using MIPS bitwise AND instruction
- Implement OR gate truth table generation using MIPS bitwise OR instruction
- Implement NOT gate truth table using arithmetic negation ($1 - A$ approach)
- Design formatted output routines for truth table display
- Implement nested loop structures for exhaustive input enumeration ($2^2 = 4$ combinations per binary gate)

Technical Justification:

This module requires solid understanding of basic MIPS instructions and control flow primitives (loops, conditionals). The systematic truth table generation demonstrates mastery of iterative computation and formatted output, serving as the foundation for more complex modules.

5.3 Module 3: Adder Circuits and Testing

Assigned to: Rana Muddasir

This module bridges fundamental gates and complex arithmetic by implementing carry propagation logic, essential for understanding multi-bit addition and computational arithmetic.

Responsibilities:

- Implement half adder logic: $\text{Sum} = A \text{ XOR } B$, $\text{Carry} = A \text{ AND } B$
- Implement full adder logic: $\text{Sum} = A \text{ XOR } B \text{ XOR } \text{Cin}$, $\text{Cout} = (A \text{ AND } B) \text{ OR } (\text{Cin AND } (A \text{ XOR } B))$
- Generate comprehensive truth tables for both adder types (4 combinations for half adder, 8 for full adder)
- Implement nested loop structures for triple-input enumeration (full adder verification)
- Develop test cases demonstrating carry propagation correctness

Technical Justification:

This module requires understanding of combinational logic composition and arithmetic circuit design. The implementation demands careful orchestration of multiple logical operations and demonstrates how simple gates combine to create functional arithmetic units.

5.4 Module 4: System Integration and I/O Management

Assigned to: Muhammad Ahmed

This module ensures cohesive system operation by managing user interaction, output formatting, and module integration across all simulation phases.

Responsibilities:

- Design and implement comprehensive user interface with phase-separated output formatting
- Implement banner generation and section headers for improved readability
- Develop string formatting routines for truth tables, ALU results, and CPU state displays
- Create main program flow orchestrating all four simulation phases
- Implement memory initialization routines for CPU simulation
- Coordinate integration testing across all modules and ensure consistent interface compliance

Technical Justification:

This module requires understanding of MIPS syscalls, string manipulation, and program structure organization. It ensures the entire system presents a cohesive, professional interface while managing the flow control that orchestrates all simulation components.

6. Expected Outcomes and Deliverables**6.1 Technical Deliverables**

- Fully functional MIPS assembly program demonstrating hierarchical processor simulation
- Comprehensive documentation including code comments, algorithm explanations, and architectural diagrams
- Verification results showing correct operation across all simulation phases
- Demonstration video showing program execution with explanation of each simulation phase

6.2 Academic Outcomes

- Demonstration of complex engineering problem-solving through multi-layered system design
- Evidence of mastery in MIPS assembly programming including advanced control flow and memory management
- Application of digital logic design principles to software implementation
- Understanding of instruction set architecture and processor microarchitecture relationships

6.3 Performance Metrics

- 100% truth table coverage for all implemented logic gates and arithmetic circuits
- Successful execution of sample programs demonstrating CPU instruction set completeness
- Correct flag generation for all ALU operations under diverse test conditions
- Zero runtime errors during program execution in MARS simulator

7. Project Timeline

Phase	Deliverables	Duration
Week 1-2	Phase 1 & 2 Implementation - Logic Gates and Adder Circuits	2 weeks
Week 3-4	Phase 3 Implementation - 8-bit ALU with Flag Generation	2 weeks
Week 5-7	Phase 4 Implementation - Mini-CPU Architecture and Execution	3 weeks
Week 8	System Integration, Testing, and Documentation	1 week
Week 9	Final Report Preparation and Demonstration Practice	1 week

8. Development Tools and Resources

8.1 Required Software

- MARS MIPS Simulator v4.5 - Primary development and testing platform

8.2 Reference Materials

- Patterson & Hennessy - Computer Organization and Design (MIPS Edition)
- MIPS Assembly Language Programming Guide - Official instruction set reference
- Course lecture notes on digital logic design and processor architecture

9. Anticipated Challenges and Mitigation Strategies

9.1 Technical Challenges

Challenge	Mitigation Strategy
Complex state management in CPU simulation using only registers	Leverage data segment memory for persistent state storage; design clear memory layout diagrams
Debugging assembly code without high-level debugging tools	Implement comprehensive diagnostic output at each phase; use MARS step-by-step execution
Module integration across team members	Define strict interface contracts; implement stub functions for early integration testing
Limited arithmetic operations in MIPS	Use bitwise operations creatively; implement helper subroutines for complex operations

10. Conclusion

This project represents a comprehensive exploration of computer architecture fundamentals through practical assembly language implementation. By constructing a hierarchical simulation spanning logic gates to a functional processor, we demonstrate the intimate connection between theoretical computer science concepts and their practical realization in executable code.

The modular approach ensures clear division of responsibilities while maintaining system cohesion, allowing each team member to develop specialized expertise while contributing to a unified deliverable. The progressive complexity of the four phases mirrors real-world engineering design methodology, where simple verified components compose into sophisticated systems.

Upon completion, this project will serve as both an educational artifact demonstrating mastery of MIPS assembly programming and a practical illustration of how processors execute instructions through coordinated interaction of datapath and control components. The verification methodology ensures correctness at each abstraction level, providing confidence in the final integrated system.

We are confident this proposal meets all requirements for a complex engineering problem and look forward to implementing this comprehensive processor simulation to deepen our understanding of computer architecture principles.

11. References

1. Bahria University. (2025). *Computer Architecture & Logic Design Lab Manual* (CEL 220). Department of Software Engineering, Karachi Campus.
2. GitHub. (n.d.). *MARS MIPS Simulator Repository*. Retrieved November 20, 2025, from https://github.com/thomasrussellmurphy/MARS_Assembler
3. MIPS Open. (n.d.). *MIPS Instruction Set Reference*. Retrieved November 20, 2025, from <https://www.mipsopenarchitecture.com/>
4. MIPS Technologies. (n.d.). *MIPS Architecture and Programmers' Manuals*. Retrieved November 20, 2025, from <https://www.mips.com/products/architectures/mips32-2/>
5. Stallings, W. (2013). *Computer Organization and Architecture: Designing for Performance* (9th ed.). Pearson Education.
6. Vollmar, P., & Sanderson, P. (2006). MARS: MIPS Assembler and Runtime Simulator - An Integrated Development Environment for MIPS Programming. *Journal of Computing Sciences in Colleges*, 21(1), 47-63.

Teacher's Signatures: _____

Remarks: _____

